



INFORME DE GUIA PRACTICA

1. Portada

Tema:	Sistema BI de segmentacion KMeans y recomendacion hibrida
Unidad de Organizacion Curricular:	Profesional.
Nivel y Paralelo:	6to - A
Alumnos participantes:	Toala Steeven Lopez Alexis
Asignatura:	Inteligencia de Negocios
Docente:	Mg. Ing. Ruben Nogales.

2. Informe

2.1. Objetivos

2.1.1. General

Desarrollar un flujo integral de analitica de datos para alquiler de peliculas que permita cargar y enriquecer informacion en MongoDB, segmentar clientes con KMeans y generar recomendaciones explicables para soporte de decisiones.

2.1.2. Especificos

- Integrar datos de hechos y dimensiones desde Excel hacia un modelo enriquecido y legible para negocio.
- Construir DataFrames agregados utiles para analisis de peliculas, categorias y perfiles de clientes.
- Implementar segmentacion de clientes usando KMeans con variables de actividad, gasto y variedad.
- Desarrollar un recomendador hibrido que combine filtrado colaborativo, contenido y ajuste por cluster.
- Mejorar la interpretabilidad mediante motivos de recomendacion y desglose del calculo del score final.
- Presentar resultados mediante interfaz grafica y visualizaciones de clusters orientadas a usuarios no tecnicos.

2.2. Introduccion

El proyecto implementa una arquitectura por capas para resolver un caso de negocio de recomendacion sobre datos de alquiler. Primero se construye un proceso ETL que toma BI-FINAL.xlsx, integra dimensiones (cliente, pelicula, categoria, tiempo, tienda, ciudad y actores) y publica un fact enriquecido en JSON y MongoDB. Como parte de la limpieza, se eliminan IDs tecnicos de la salida analitica para trabajar con referencias legibles.

Sobre esta base se generan DataFrames agregados desde MongoDB: resumen de peliculas, resumen de categorias y perfil de clientes. Estos artefactos alimentan dos componentes clave:



- Segmentacion de clientes con KMeans en `logica/clusters.py`.
- Recomendacion hibrida en `logica/recomendacion.py`.

El recomendador combina once senales ponderadas, entre ellas similitud coseno cliente-cliente, afinidad por categoria, popularidad, ajuste temporal y ajuste por segmento KMeans. El score final se calcula como suma ponderada:

$$\text{score_final} = \sum_{i=1}^{11} w_i s_i$$

Con la implementacion actual:

$$\begin{aligned} \text{score_final} = & 0,30 s_{\text{colaborativo}} + 0,16 s_{\text{categoria}} + 0,11 s_{\text{popularidad}} + 0,07 s_{\text{ingreso}} \\ & + 0,05 s_{\text{duracion}} + 0,08 s_{\text{categoria_global}} + 0,07 s_{\text{ajuste_cliente}} \\ & + 0,04 s_{\text{temporal}} + 0,02 s_{\text{diversidad}} + 0,06 s_{\text{cluster_ajuste}} + 0,04 s_{\text{cluster_categoria}} \end{aligned}$$

En la interfaz Tkinter, el usuario puede seleccionar cliente, generar recomendaciones, visualizar motivo principal y revisar el detalle matematico del score por pelicula. En paralelo, la vista de clusters entrega graficos en ventanas separadas con descripciones para facilitar interpretacion.

2.3. Desarrollo tecnico detallado del sistema

2.3.1. 1) Proceso ETL completo: desde Excel hasta JSON y MongoDB

El flujo inicia en `logica/carga_datos.py` con el archivo `BI-FINAL.xlsx`. Este modulo tiene como objetivo transformar una estructura transaccional con IDs tecnicos en un fact enriquecido, legible y listo para analitica.

Paso 1: lectura de hojas y control de existencia Se utiliza `pd.ExcelFile` para leer una sola vez el libro y luego extraer cada hoja requerida con la funcion `cargar_hoja`. Si una hoja obligatoria no existe, el proceso se detiene con error explicito.

```
if nombre_hoja not in excel.sheet_names:
    raise ValueError(f'No existe la hoja requerida: {nombre_hoja}')
```

Paso 2: limpieza tipologica del fact En `preparar_fact` se convierten a numerico columnas de medida y claves (`ingreso`, `duracion_alquiler`, `id_*`) usando `errors='coerce'` para capturar inconsistencias sin romper el flujo. Posteriormente se filtran registros sin `ingreso`, porque el sistema de recomendacion y los agregados dependen de este campo.

Paso 3: enriquecimiento dimensional Con una secuencia de `merge` por llave, el fact incorpora atributos descriptivos:

- Cliente: nombre, apellido, email, estado activo.
- Pelicula: titulo, duracion, clasificacion, idioma, precio.
- Categoria, ciudad, tiempo y tienda.

Esto convierte datos orientados a almacenamiento en datos orientados a analisis y explicacion de negocio.



Paso 4: consolidacion de actores por pelicula Con la tabla puente PUENTE_PELICULA_ACTOR y DIM_ACTOR se construyen:

- `pelicula_actores`: lista de actores unicos por pelicula.
- `cantidad_actores`: total de actores unicos.

La agregacion usa `groupby('id_pelicula')` para concentrar la informacion en un solo registro por pelicula.

Paso 5: identificador legible de cliente Se crea `cliente_nombre_completo` concatenando nombre y apellido, para evitar depender de IDs internos en reportes e interfaz.

Paso 6: salida final y persistencia Antes de guardar, se eliminan columnas que inician con `id_`. El resultado se persiste en:

- JSON: `fact_alquiler_proyecto.json`.
- MongoDB: `BI_Final.fact_alquiler`.

Se ejecuta `collection.drop()` para evitar duplicados entre corridas y garantizar consistencia de una carga completa por ejecucion.

2.3.2. 2) Como se construyen y para que sirven los DataFrames

La capa logica/`dataframes.py` construye DataFrames agregados desde MongoDB para separar calculo analitico de la capa de interfaz.

DataFrame 1: `df_resumen_peliculas` Se agrupa por `pelicula_titulo` y se calculan:

- `total_alquileres`: cuantas veces se alquilo.
- `ingreso_promedio` y `ingreso_total`: rentabilidad.
- `clientes_unicos`: alcance real de audiencia.
- `duracion_promedio`: comportamiento temporal del alquiler.

Por que se usa: es la tabla base para rankear candidatos en recomendacion, porque contiene popularidad, valor economico y comportamiento de consumo.

DataFrame 2: `df_resumen_categorias` Se agrupa por `categoria_nombre` y se obtiene volumen, clientes e ingresos. **Por que se usa:** permite medir fuerza global de categoria y evitar que una recomendacion dependa solo de similitud cliente-cliente.

DataFrame 3: `df_perfil_clientes` Se agrupa por `cliente_ref` y se calcula:

- `total_alquileres` (actividad),
- `peliculas_unicas` y `categorias_unicas` (variedad),
- `gasto_total` y `gasto_promedio` (capacidad de gasto).

Por que se usa: es el vector de comportamiento que alimenta KMeans y tambien componentes del recomendador.



Normalizaciones aplicadas Para combinar metricas con escalas distintas se usa:

$$\hat{x} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Si $\max(x) = \min(x)$, la serie se reemplaza por ceros para evitar division por cero.

En categorias se define un score combinado:

$$\text{score_categoria_global} = 0,6 \cdot \text{popularidad_categoria_norm} + 0,4 \cdot \text{ingreso_categoria_norm}$$

2.3.3. 3) Algoritmo de clusters: como se obtienen exactamente

La segmentacion esta en `logica/clusters.py` y usa KMeans sobre `df_perfil_clientes`.

Variables de entrada al modelo

- `gasto_promedio`
- `total_alquileres`
- `peliculas_unicas`
- `categorias_unicas`

Pipeline del clustering

1. Conversion de columnas a numerico y eliminacion de filas invalidas.
2. Estandarizacion con `StandardScaler` para que ninguna variable domine por escala.
3. Definicion de k robusta: `k = min(n_clusters, numero_clientes_validos)`.
4. Entrenamiento de `KMeans(random_state=42, n_init=10)`.
5. Etiquetado semantico de clusters por gasto promedio del centroide.

Interpretacion de etiquetas Los clusters no se dejan como numeros tecnicos. Se renombran como:

- Mayor gasto
- Gasto medio
- Menor gasto

segun el orden descendente del centro de `gasto_promedio`.

Salida final de segmentacion Cada cliente queda con: `cliente_ref`, `cluster_id`, `cluster_nombre`, `cluster_gasto_centro`. Esta salida se integra al recomendador y tambien se exporta en `clusters_fact_alquiler_proyect`

Fragmento representativo del codigo aplicado:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(base[columnas_modelo].fillna(0.0))

kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
base['cluster_id'] = kmeans.fit_predict(X_scaled)
```



2.3.4. 4) Algoritmo del sistema de recomendacion: desglose completo

El modulo logica/recomendacion.py implementa un recomendador hibrido de 11 senales.

EtapA A: preparacion y matrices base

- Se construye `cliente_ref` y `pelicula_ref` en formato legible.
- Se arma matriz cliente-pelicula por conteo de alquileres.
- Se separan peliculas vistas y candidatas no vistas.

EtapA B: similitud colaborativa Se calcula similitud coseno cliente-cliente:

$$\cos(\theta) = \frac{u \cdot v}{\|u\| \|v\|}$$

Con los `n_vecinos` mas similares se estima una preferencia ponderada por pelicula candidata.

EtapA C: senales de contenido y contexto Para cada candidato se calculan:

- `score_categoria`: afinidad del cliente con la categoria.
- `score_categoria_global`: fortaleza global de la categoria.
- `score_popularidad`, `score_ingreso`, `score_duracion` desde resumen de peliculas.
- `score_ajuste_cliente`: cercania entre ingreso promedio de pelicula y gasto promedio del cliente.
- `score_temporal`: similitud entre patron mensual del cliente y patron mensual de la pelicula.
- `score_diversidad`: incentivo a categorias menos consumidas por ese cliente.

EtapA D: senales de cluster Usando la segmentacion KMeans:

- `score_cluster_ajuste`: cercania entre ingreso de pelicula y centro economico del cluster.
- `score_cluster_categoria`: frecuencia de la categoria dentro del mismo cluster del cliente.

EtapA E: score final y ranking El ranking se obtiene por suma ponderada:

$$\begin{aligned} \text{score_final} = & 0,30 s_{\text{colaborativo}} + 0,16 s_{\text{categoria}} + 0,11 s_{\text{popularidad}} + 0,07 s_{\text{ingreso}} \\ & + 0,05 s_{\text{duracion}} + 0,08 s_{\text{categoria_global}} + 0,07 s_{\text{ajuste_cliente}} \\ & + 0,04 s_{\text{temporal}} + 0,02 s_{\text{diversidad}} + 0,06 s_{\text{cluster_ajuste}} + 0,04 s_{\text{cluster_categoria}} \end{aligned}$$

Fragmento representativo del codigo aplicado:

```
candidatos['score_final'] = (  
    candidatos['score_colaborativo'] * 0.30  
    + candidatos['score_categoria'] * 0.16  
    + candidatos['score_popularidad'] * 0.11  
    + candidatos['score_ingreso'] * 0.07  
    + candidatos['score_duracion'] * 0.05  
    + candidatos['score_categoria_global'] * 0.08  
    + candidatos['score_ajuste_cliente'] * 0.07
```



```
+ candidatos['score_temporal'] * 0.04  
+ candidatos['score_diversidad'] * 0.02  
+ candidatos['score_cluster_ajuste'] * 0.06  
+ candidatos['score_cluster_categoria'] * 0.04  
)
```

Luego se ordena de mayor a menor `score_final` y se devuelven las `n_recomendaciones` top.

Etapas F: explicabilidad Cada fila recomendada incorpora:

- **motivo:** explicacion corta para lectura rapida.
- **motivo_detalle:** desglose numerico tipo `valor x peso = aporte` por cada senal.

Esto permite auditar el por que de cada resultado.

2.3.5. 5) Como funciona la interfaz del sistema

La vista `vistas/interfaz_recomendacion.py` conecta usuario y motor de recomendacion.

Flujo de uso

1. Carga inicial desde MongoDB.
2. Seleccion de cliente en Combobox.
3. Definicion de numero de recomendaciones y vecinos.
4. Ejecucion de `recomendar_peliculas()`.
5. Visualizacion en tabla (pelicula, categoria, score, motivo principal).
6. Panel inferior con detalle matematico de la recomendacion seleccionada.

Elementos de salida para el usuario

- **Resumen del cliente:** algoritmo, segmento, historico, vecino mas similar, categorias preferidas.
- **Tabla principal:** resultado compacto para decision rapida.
- **Detalle de score:** trazabilidad completa de aportes.
- **Exportacion CSV:** persistencia externa de recomendaciones.

2.3.6. 6) Visualizacion de clusters

La descripcion puntual de cada grafico se presenta junto a su imagen correspondiente en la seccion de evidencia grafica, para facilitar la lectura directa de cada resultado.



2.3.7. 7) Integración end-to-end del sistema

El sistema opera con una trazabilidad clara:

1. ETL integra y limpia Excel.
2. Se genera JSON y se actualiza MongoDB.
3. Desde Mongo se construyen DataFrames analíticos.
4. Con perfil de clientes se ejecuta KMeans.
5. El recomendador combina colaborativo + contenido + contexto + cluster.
6. Interfaz y gráficos exponen resultados con explicabilidad.

2.3.8. 8) Robustez y controles incluidos

Se incorporaron validaciones para mantener estabilidad:

- validación de hojas requeridas en origen,
- conversión segura de tipos con `errors='coerce'`,
- manejo de dataset vacío en cada capa,
- protección ante división por cero en normalizaciones,
- fallback de identificadores legibles cuando faltan columnas técnicas,
- mensajes de error orientados a diagnóstico.

2.3.9. 9) Fragmentos de implementación clave (código real)

Como el evaluador puede no tener acceso al repositorio, se incluyen fragmentos centrales de la implementación para respaldar técnicamente el informe.

ETL: eliminación de IDs técnicos y persistencia dual

```
def guardar_json_y_mongo(df: pd.DataFrame) -> None:
    columnas_sin_ids = [
        col for col in df.columns
        if not col.lower().startswith('id_')
    ]
    df_salida = df[columnas_sin_ids]
    registros = df_salida.to_dict(orient='records')

    with open(RUTA_JSON, 'w', encoding='utf-8') as archivo:
        json.dump(registros, archivo, ensure_ascii=False, indent=4)

    client = MongoClient(MONGO_URI)
    db = client[DB_NAME]
    collection = db[COLLECTION_NAME]
    collection.drop()
    if registros:
        collection.insert_many(registros)
```



Carga inicial desde Excel y validacion de hojas requeridas

```
def cargar_hoja(excel: pd.ExcelFile, nombre_hoja: str) -> pd.DataFrame:
    if nombre_hoja not in excel.sheet_names:
        raise ValueError(f'No existe la hoja requerida: {nombre_hoja}')
    return excel.parse(nombre_hoja)

def leer_tablas() -> dict:
    excel = pd.ExcelFile(RUTA_EXCEL)
    tablas = {
        'fact': cargar_hoja(excel, 'FACT_ALQUILER'),
        'actor': cargar_hoja(excel, 'DIM_ACTOR'),
        'categoria': cargar_hoja(excel, 'DIM_CATEGORIA'),
        'ciudad': cargar_hoja(excel, 'DIM_CIUADAD'),
        'cliente': cargar_hoja(excel, 'DIM_CLIENTE'),
        'pelicula': cargar_hoja(excel, 'DIM_PELICULA'),
        'tiempo': cargar_hoja(excel, 'DIM_TIEMPO'),
        'tienda': cargar_hoja(excel, 'DIM_TIENDA'),
        'puente': cargar_hoja(excel, 'PUENTE_PELICULA_ACTOR'),
    }
    return tablas
```

Limpieza de campos numericos en la carga del fact

```
def preparar_fact(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    columnas_numericas = [
        'id_hecho', 'id_tiempo', 'id_cliente', 'id_pelicula',
        'id_tienda', 'id_categoria', 'id_ciudad',
        'cantidad_alquiler', 'ingreso', 'duracion_alquiler'
    ]
    for columna in columnas_numericas:
        if columna in df.columns:
            df[columna] = pd.to_numeric(df[columna], errors='coerce')

    df = df.dropna(subset=['ingreso'])
    return df
```

Carga de datos para el recomendador desde MongoDB

```
def cargar_datos_mongo() -> pd.DataFrame:
    client = MongoClient(MONGO_URI)
    db = client[DB_NAME]
    collection = db[COLLECTION_NAME]
    df = pd.DataFrame(list(collection.find({}, {'_id': 0})))
    if df.empty:
        raise ValueError('No hay datos cargados en MongoDB para recomendar.')
    return df

def preparar_datos(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df['ingreso'] = pd.to_numeric(df['ingreso'], errors='coerce')
```




```
if 'cliente_nombre_completo' in df.columns:
    df['cliente_ref'] = df['cliente_nombre_completo'].astype(str).str.strip()
if 'pelicula_titulo' in df.columns:
    df['pelicula_ref'] = df['pelicula_titulo'].astype(str).str.strip()

df = df.dropna(subset=['cliente_ref', 'pelicula_ref', 'ingreso'])
return df
```

Agregacion en MongoDB para peliculas

```
resultado_2 = list(fact_alquiler.aggregate([
    {'$group': {
        '_id': '$pelicula_titulo',
        'pelicula_titulo': {'$first': '$pelicula_titulo'},
        'categoria_nombre': {'$first': '$categoria_nombre'},
        'total_alquileres': {'$sum': 1},
        'ingreso_promedio': {'$avg': '$ingreso'},
        'ingreso_total': {'$sum': '$ingreso'},
        'clientes_unicos': {
            '$addToSet': {
                '$ifNull': [
                    '$cliente_nombre_completo',
                    '$cliente_nombre'
                ]
            }
        }
    }
}],
{'$project': {
    '_id': 0,
    'pelicula_ref': '$pelicula_titulo',
    'pelicula_titulo': '$pelicula_titulo',
    'categoria_nombre': {'$ifNull': ['$categoria_nombre', 'Sin categoria']},
    'total_alquileres': 1,
    'ingreso_promedio': 1,
    'ingreso_total': 1,
    'clientes_unicos': {'$size': '$clientes_unicos'}
}}
]))
```

Construccion de matriz cliente-pelicula

```
def construir_matriz_clientes_peliculas(df: pd.DataFrame) -> pd.DataFrame:
    return pd.pivot_table(
        df,
        index='cliente_ref',
        columns='pelicula_ref',
        values='ingreso',
        aggfunc='count',
        fill_value=0,
    )
```

Similitud coseno cliente-cliente

```
def calcular_similitud_coseno(matriz: pd.DataFrame, cliente_id: str) -> pd.Series:
    vector_objetivo = matriz.loc[cliente_id].to_numpy(dtype=float)
```



```
norma_objetivo = np.linalg.norm(vector_objetivo)
similitudes = {}

for otro_cliente, fila in matriz.iterrows():
    if otro_cliente == cliente_id:
        continue
    vector_otro = fila.to_numpy(dtype=float)
    norma_otro = np.linalg.norm(vector_otro)
    if norma_objetivo > 0 and norma_otro > 0:
        similitud = float(
            np.dot(vector_objetivo, vector_otro)
            / (norma_objetivo * norma_otro)
        )
        if similitud > 0:
            similitudes[str(otro_cliente)] = similitud

return pd.Series(similitudes).sort_values(ascending=False)
```

Motivo detallado y trazabilidad del score

```
def construir_motivo_detallado(fila: pd.Series) -> str:
    factores = [
        ('Colaborativo (clientes similares)', 'score_colaborativo', 0.30),
        ('Afinidad por categoria', 'score_categoria', 0.16),
        ('Popularidad de la pelicula', 'score_popularidad', 0.11),
        ('Ingreso promedio de la pelicula', 'score_ingreso', 0.07),
        ('Duracion promedio', 'score_duracion', 0.05),
    ]
    lineas = ['Calculo del score final (suma ponderada):']
    for nombre, col, peso in factores:
        valor = float(fila.get(col, 0.0))
        aporte = valor * peso
        lineas.append(f"- {nombre}: {valor:.4f} x {peso:.2f} = {aporte:.4f}")
    return '\\n'.join(lineas)
```

Interfaz: carga de resultados en tabla

```
for idx, fila in recomendaciones.reset_index(drop=True).iterrows():
    self.tree.insert(
        '',
        tk.END,
        iid=str(idx),
        values=(
            fila.get('pelicula_titulo', ''),
            fila.get('categoria_nombre', ''),
            f"{float(fila.get('score_final', 0.0)):.4f}",
            self._resumir_motivo(fila.get('motivo', '')),
        ),
    )
```

2.4. Metodología aplicada

El desarrollo se ejecuto de manera incremental, validando cada capa antes de avanzar a la siguiente para evitar errores acumulados.



2.4.1. Fase 1: Integracion y calidad de datos

- Se definio un flujo ETL para convertir datos de origen en un fact enriquecido.
- Se normalizaron tipos numericos y se descartaron registros invalidos para evitar ruido en el modelado.
- Se reemplazaron identificadores tecnicos por referencias legibles para mejorar comprension en reportes y UI.

2.4.2. Fase 2: Modelado analitico

- Se construyeron agregados de peliculas, categorias y clientes como capa semantica reusable.
- Se aplico KMeans sobre variables estandarizadas para evitar sesgo por escala.
- Se etiquetaron segmentos por gasto para hacer la segmentacion interpretable en lenguaje de negocio.

2.4.3. Fase 3: Motor de recomendacion

- Se construyo matriz cliente-pelicula para similitud colaborativa.
- Se incorporaron senales de contenido y contexto temporal para reducir recomendaciones triviales.
- Se incluyo ajuste por cluster para alinear recomendaciones al perfil de segmento.

2.4.4. Fase 4: Explicabilidad y visualizacion

- Se implemento motivo principal y desglose de aportes por componente del score.
- Se mejoraron textos y etiquetas de graficos para usuarios no tecnicos.
- Se separaron visualizaciones por ventana para facilitar lectura y comparacion.

2.5. Arquitectura y flujo operacional

1. **Entrada:** archivo BI-FINAL.xlsx con tabla de hechos y dimensiones.
2. **ETL:** enriquecimiento y persistencia en JSON + MongoDB.
3. **Analitica base:** calculo de DataFrames agregados desde MongoDB.
4. **Segmentacion:** agrupacion de clientes mediante KMeans.
5. **Recomendacion:** ranking hibrido con score ponderado.
6. **Presentacion:** interfaz Tkinter y graficos de interpretacion.

2.6. Validaciones realizadas

- Verificacion de carga correcta en MongoDB y consistencia del numero de registros.
- Validacion de columnas criticas para clustering y recomendacion.
- Revision de ejecutabilidad de scripts en `logica/` y `vistas/`.
- Comprobacion de explicaciones de score y legibilidad de interfaz.



2.7. Resultados obtenidos

- Pipeline completo funcional: Excel → JSON/MongoDB → Clusters → Recomendaciones.
- Segmentacion estable y reusable para analisis y para mejorar el score del recomendador.
- Recomendaciones explicables, con trazabilidad de cada componente del score.
- Interfaz mejorada en legibilidad de motivos y detalle de calculo.
- Graficos de cluster reetiquetados para usuarios sin perfil tecnico.

2.8. Evidencia grafica de resultados

En esta seccion se dejan espacios para insertar capturas reales de la ejecucion del sistema.

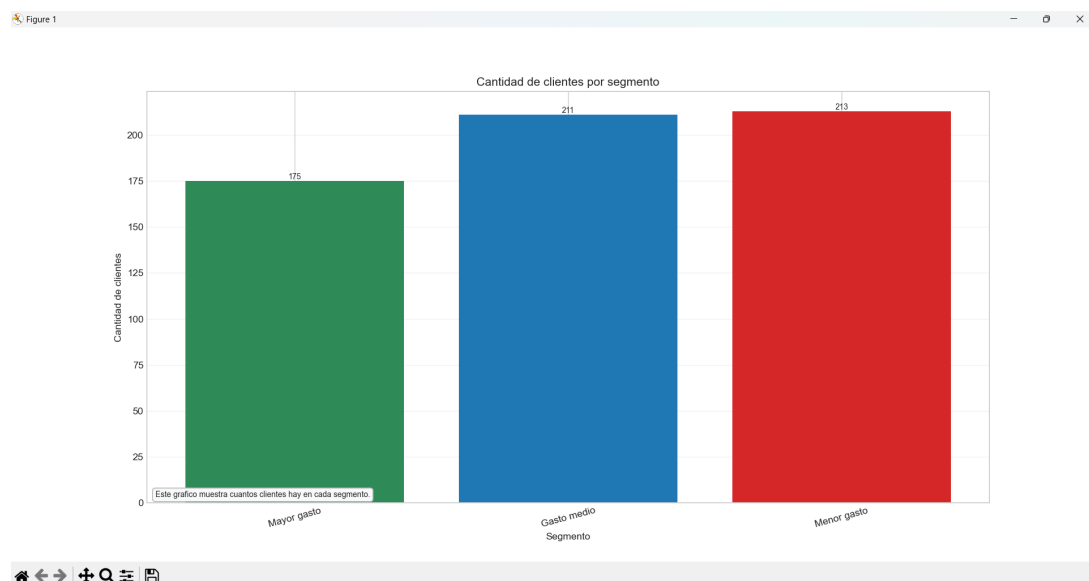


Figura 1: Distribucion de clientes por segmento

Descripcion: Este grafico muestra el tamano relativo de cada cluster. La altura de cada barra representa la cantidad de clientes del segmento y permite identificar que grupos son predominantes y en cuales conviene priorizar estrategias comerciales.

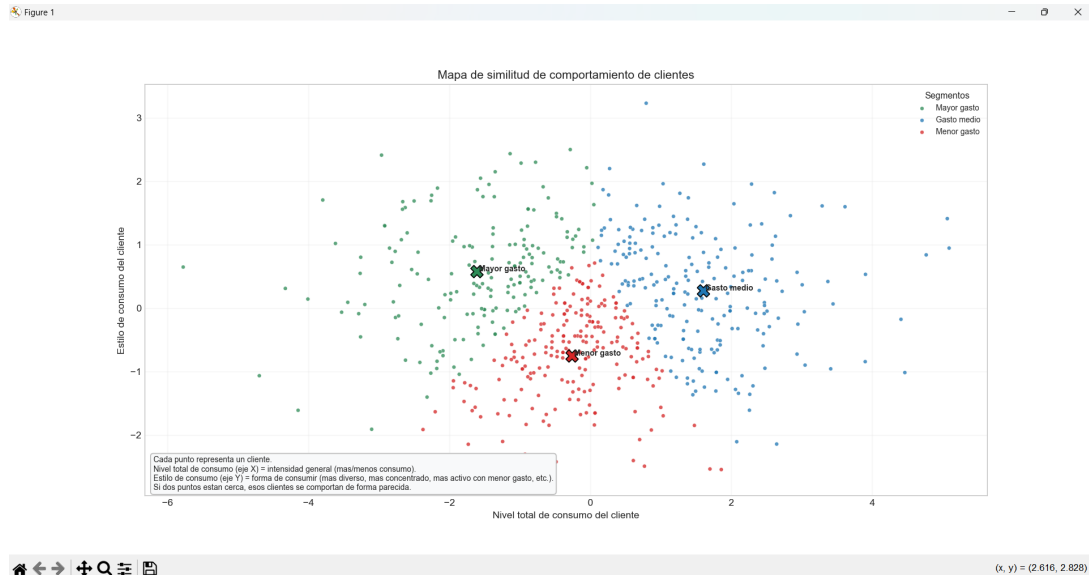


Figura 2: Mapa de similitud de comportamiento de clientes

Descripcion: Esta visualización proyecta a los clientes en 2 dimensiones mediante PCA usando variables de actividad, gasto y variedad. Cada punto representa un cliente; puntos cercanos indican comportamientos similares. El tamaño del punto está asociado a `gasto_promedio`.

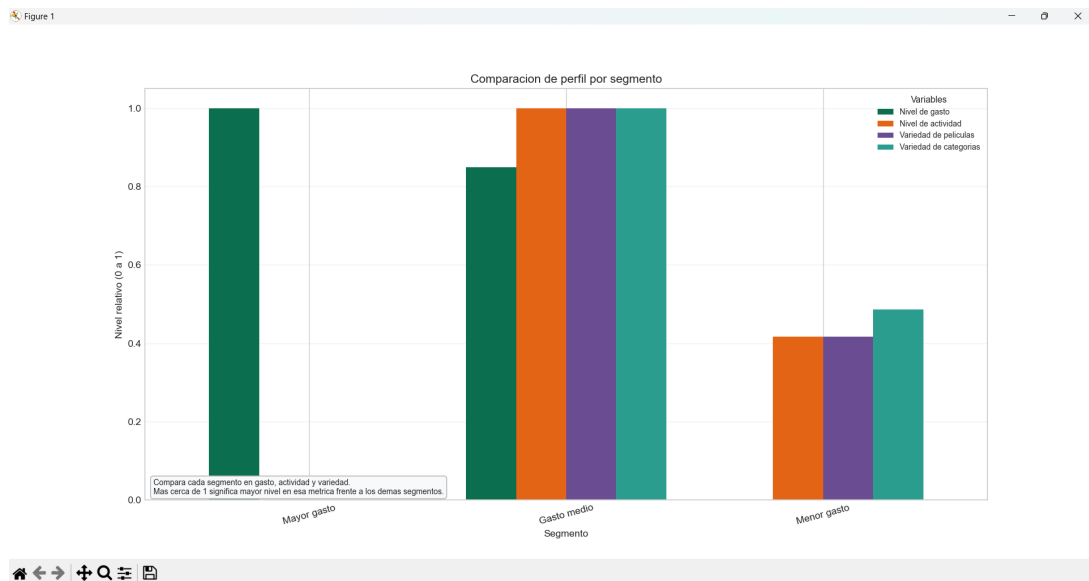


Figura 3: Comparacion de perfil por segmento

Descripcion: Este gráfico compara el perfil medio de cada segmento en escala normalizada [0, 1]: nivel de gasto, nivel de actividad, variedad de películas y variedad de categorías. Permite observar rápidamente fortalezas relativas de cada cluster sin sesgo por diferencias de escala.

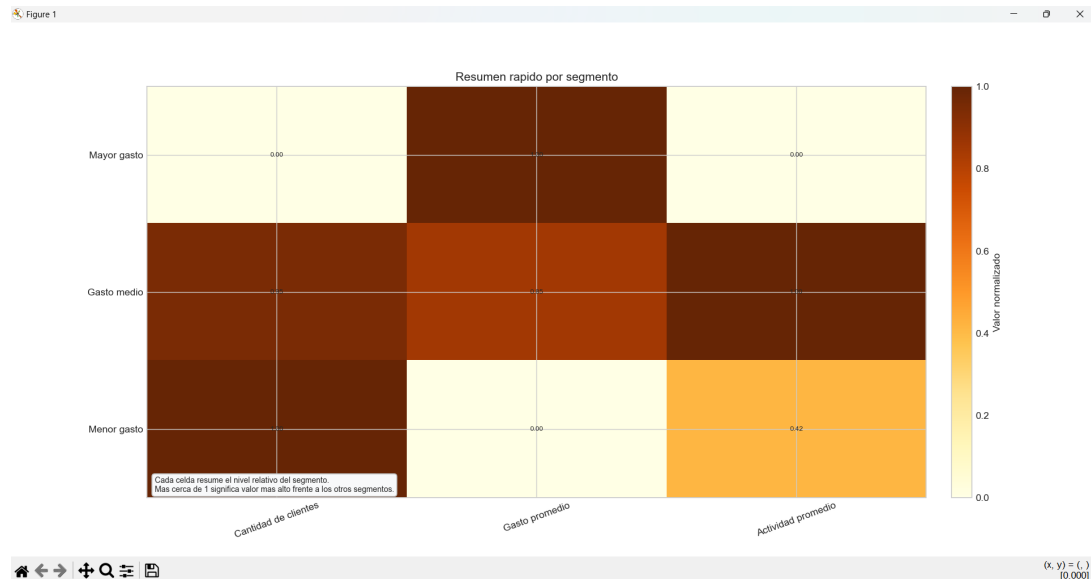


Figura 4: Resumen rapido por segmento

Descripcion: El mapa de calor resume por segmento la cantidad de clientes, el gasto promedio y la actividad promedio en valores normalizados. Los colores mas intensos representan niveles mas altos y facilitan comparar el peso comercial de cada cluster en una sola vista.

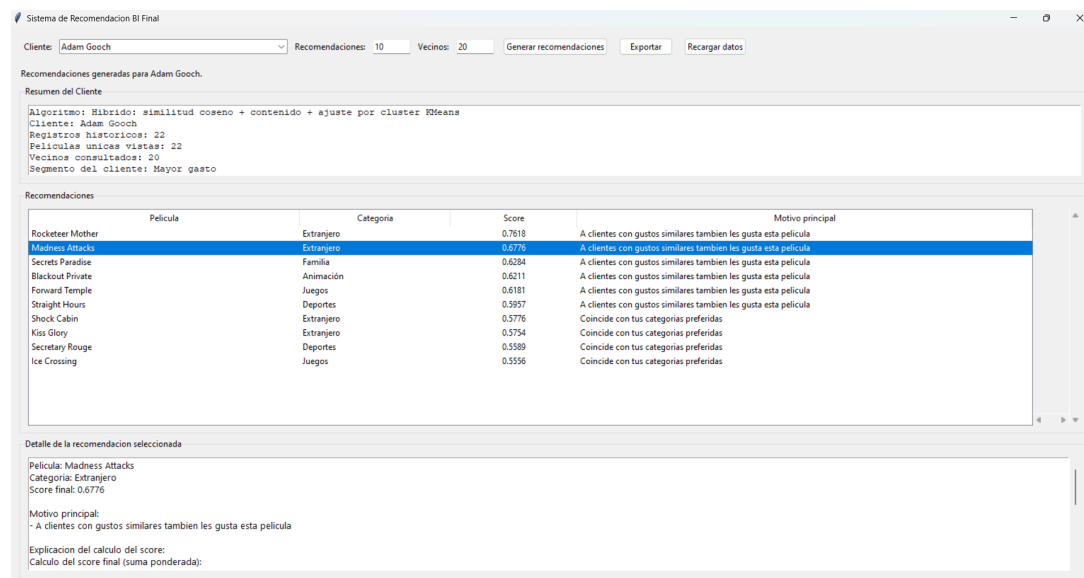


Figura 5: Interfaz de recomendacion con detalle de score

Descripcion: Este grafico muestra la interfaz del sistema de recomendacion con seleccion de cliente, tabla de peliculas recomendadas y panel de detalle explicativo. En ese panel se desglosa el **score final** por componentes, lo que permite interpretar por que una pelicula fue priorizada.



2.9. Conclusiones

- Se logro una solucion BI integral, coherente y mantenible al separar responsabilidades entre modulos de logica y vistas.
- El uso de KMeans sobre perfiles de cliente aporta valor tanto analitico como operativo dentro del sistema de recomendacion.
- La explicabilidad del score final mejora la confianza en las recomendaciones y facilita su validacion por parte de usuarios de negocio.
- El ajuste de etiquetas y textos en visualizaciones fue clave para que la interpretacion no dependa de conocimientos tecnicos de programacion o modelado.

2.10. Recomendaciones

- Incorporar seleccion automatica de numero de clusters (metodo codo/silhouette) para robustecer el modelado.
- Medir desempeno del recomendador con metricas offline como Precision@K y Recall@K.
- Agregar historico de ejecuciones para comparar evolucion de segmentos y cambios de comportamiento.
- Estandarizar la plantilla de reportes del proyecto para futuras iteraciones y evidencias academicas.

2.11. Referencias

Referencias

- [1] scikit-learn Developers, *scikit-learn User Guide*, https://scikit-learn.org/stable/user_guide.html
- [2] The pandas development team, *pandas Documentation*, <https://pandas.pydata.org/docs/>
- [3] MongoDB Inc., *PyMongo Documentation*, <https://pymongo.readthedocs.io/>
- [4] Matplotlib Development Team, *Matplotlib Documentation*, <https://matplotlib.org/stable/>